



Katholieke  
Universiteit  
Leuven

Department of  
Electrical Engineering

## **PROJECT 2: ANALYZING A CRYPTOSYSTEM BASED ON THE FINITE FIELD ISOMORPHISM PROBLEM**

Computer Algebra for Cryptography (B-KUL-H0E74A)

**Gust De Wit (r0948039)**

Academic year 2025–2026

# 1 Finite Fields and Irreducible Polynomials

## 1.1 Finding an isomorphism

### 1.1.1 Alternate formulation

First, we will show that finding an isomorphism between  $\mathbb{X}\langle x \rangle = \mathbb{F}_p[z]/f(z)$  and  $\mathbb{Y}\langle y \rangle = \mathbb{F}_p[z]/g(z)$  is equivalent to finding a single root of  $f(z)$  inside  $\mathbb{Y}$ .

Since  $f$  is monic of degree  $n$  over  $\mathbb{F}_p$ , we can write  $f(z) = z^n + c_{n-1}z^{n-1} + \dots + c_0, \forall c_i \in \mathbb{F}_p$ . Furthermore, we know that an isomorphism  $\varphi$  is a ring homomorphism and fixes  $\mathbb{F}_p$ . Hence, for any element  $\alpha \in \mathbb{X}$ :

$$\varphi(f(\alpha)) = \varphi(\alpha^n + c_{n-1}\alpha^{n-1} + \dots + c_0) = \varphi(\alpha)^n + c_{n-1}\varphi(\alpha)^{n-1} + \dots + c_0 = f(\varphi(\alpha)). \quad (1)$$

Applying this to  $\alpha = x$  and using  $f(x) = 0$  by definition of  $\mathbb{X}$ , we get  $0 = \varphi(0) = \varphi(f(x)) = f(\varphi(x))$  in  $\mathbb{Y}$ . **Hence  $\varphi(x)$  is a root of  $f$  in  $\mathbb{Y}$ .**

“Conversely, let  $y_0 \in \mathbb{Y}$  be any root of  $f$ . Every element of  $\mathbb{X}$  can be written uniquely as  $a(x)$  for a polynomial  $a$  of degree  $< n$  over  $\mathbb{F}_p$ ; define  $\varphi(a(x)) = a(y_0)$  for every such element. This map is well defined (if  $f \mid a - b$  then  $a(y_0) - b(y_0) = 0$  since  $f(y_0) = 0$ ), it is a ring homomorphism, and it is injective since  $\mathbb{X}$  is a field; as  $|\mathbb{X}| = |\mathbb{Y}| = p^n$ , it is a bijection, hence an isomorphism. Therefore determining an isomorphism is exactly determining one root  $y_0$  of  $f$  in  $\mathbb{Y}$ .”

### 1.1.2 Existence and number of roots of $f$ in $\mathbb{Y}$

By construction  $\mathbb{X} = \mathbb{F}_p[z]/(f)$  is  $\mathbb{F}_{p^n}$ , a degree- $n$  Galois extension of  $\mathbb{F}_p$  with cyclic group generated by the Frobenius  $\sigma(u) = u^p$ . The generator  $x$  satisfies  $f(x) = 0$  with  $f$  monic irreducible of degree  $n$ , so  $f$  is its minimal polynomial; since  $\deg f = n$ ,  $x$  lies in no proper subfield, hence  $f = \prod_{i=0}^{n-1} (z - \sigma^i(x)) = \prod_{i=0}^{n-1} (z - x^{p^i})$ . The  $n$  conjugates are distinct (otherwise  $x$  would lie in a proper subfield), so  $f$  has exactly  $n$  distinct roots, all in  $\mathbb{F}_{p^n}$ . As  $\mathbb{Y} \cong \mathbb{F}_{p^n}$ ,  $f$  likewise splits completely over  $\mathbb{Y}$  into  $n$  distinct linear factors, so the algorithm only ever extracts linear factors.

### 1.1.3 Algorithm

Since all of the factors are linear, we can skip the square-free factorisation and distinct-degree factorisation steps and move straight to the equal-degree factorisation step (Cantor-Zassenhaus algorithm), with the factor degree known to be 1.

---

**Algorithm 1** FINDROOT( $f$ ) — one root of  $f$  in  $\mathbb{Y}$

---

**Require:** monic  $f \in \mathbb{Y}[w]$  splitting into distinct linear factors over  $\mathbb{Y}$

**Ensure:** a root  $y_0 \in \mathbb{Y}$  of  $f$

```

1: while  $\deg f > 1$  do
2:   repeat
3:      $h \leftarrow \text{CANTORZASSENHAUS}(f)$ 
4:   until  $h \neq \text{FAILURE}$ 
5:    $f \leftarrow h$  ▷ descend into one proper factor
6: end while
7: make  $f$  monic; write  $f = w - y_0$ 
8: return  $y_0$ 
```

---

This algorithm repeatedly calls the Cantor-Zassenhaus algorithm as described on slide 27 of the lecture notes, without the extra step of calculating the gcd because in our case, we know all roots like in  $\mathbb{F}_q$ . Each subsequent call to Cantor-Zassenhaus returns a factor of the submitted  $f(x)$  or FAILURE with chance  $\leq 1/2$ . This factor is not guaranteed to be linear yet, so we repeat this procedure until it is. At the end, we make the  $f_i$  monic and calculate  $y_0$ .

### 1.1.4 Complexity

Let  $M(n)$  be the cost in  $\mathbb{F}_p$ -operations of multiplying two degree- $< n$  polynomials over  $\mathbb{F}_p$ . One multiplication in  $\mathbb{Y}$  costs  $O(M(n))$ , so one in  $\mathbb{Y}[w]/(f)$  costs  $O(M(n)^2)$ . Root finding over a field of size  $Q$  costs  $O(M(n) \log n \log(nQ))$  operations in that field; here the field is  $\mathbb{Y}$  with  $Q = p^n$ , so  $\log(nQ) = \Theta(n \log p)$ . Converting each  $\mathbb{Y}$ -operation to  $\mathbb{F}_p$ -operations (a factor  $O(M(n))$ ) yields

$$O(n \log p \cdot \log n \cdot M(n)^2)$$

$\mathbb{F}_p$ -operations, i.e. polynomial in  $n$  and  $\log p$ .

### 1.1.5 Finding other isomorphisms

The cost of finding other isomorphisms is almost free once we have one root  $y_0$ , as the remaining  $n - 1$  roots of  $f$  in  $\mathbb{Y}$  are its Frobenius conjugates  $y_0, y_0^p, \dots, y_0^{p^{n-1}}$ . This requires only exponentiation.

## 1.2 Implementation of FieldIsomorphism(f, g)

An implementation of FIELDISOMORPHISM can be found in `field_iso.m`. The results of running the program with  $p = 2^{16} + 7$  and  $n = 2^k$  for  $k = 2, \dots, 6$  are reported in Table 1.

| $n$      | 4     | 8     | 16    | 32    | 64     |
|----------|-------|-------|-------|-------|--------|
| time (s) | 0.000 | 0.020 | 0.090 | 0.860 | 10.110 |

Table 1: Running time of FIELDISOMORPHISM for  $p = 2^{16} + 7$  and  $n = 2^k$ ,  $k = 2, \dots, 6$ .

## 1.3 Implementation of InverseFieldIsomorphism(f, g, s)

An implementation of INVERSEFIELDISOMORPHISM can be found in `field_iso.m`. This algorithm relies on the fact that  $\varphi$  is not only a field isomorphism, but also an  $\mathbb{F}_p$ -linear map between the two  $\mathbb{F}_p$ -vector spaces  $\mathbb{X}$  and  $\mathbb{Y}$  with basis  $1, x, x^2, \dots, x^{n-1}$  and  $1, y, y^2, \dots, y^{n-1}$  respectively. Since this is a linear map, defined by what it does to a basis, it can be described as a matrix.

We know  $\varphi = s(y) = y_0$ , and since  $\varphi$  is a ring homomorphism,  $\varphi(x^i) = \varphi(x)^i = y_0^i$ . So the image of the  $i$ -th basis vector of  $\mathbb{X}$  is  $y_0^i \in \mathbb{Y}$ . We can write each  $y_0^i$  as its coordinate vector in  $\mathbb{Y}$ 's basis, resulting in an  $n \times n$  matrix  $M$  over  $\mathbb{F}_p$ .

Since  $\varphi$  is a bijection,  $M$  is invertible, and  $M^{-1}$  is the matrix of the inverse map  $\psi = \varphi^{-1}$ . Applying  $M^{-1}$  to the generator  $y$  yields a polynomial  $t$  in  $\mathbb{X}$ 's basis, with  $\psi(y) = t(x)$ .

## 2 A Cryptosystem Based on the FFIP

The system as described in the assignment was implemented in `ffip.m`.

## 3 Attacking the FFIP Cryptosystem (the stupid way)

## 4 Breaking Search FFIP (the clever way)